

**Educação
Profissional
Paulista**

Técnico em
**Ciência de
Dados**

Variáveis e tipos de dados

Strings

Aula 2

Código da aula: [DADOS]ANO1C2B1S8A2

Exposição



Objetivos da aula

- Conhecer algumas operações de *strings*.



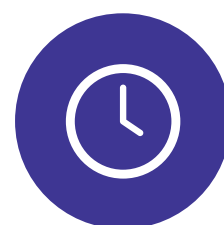
Competências da Unidade (técnicas e socioemocionais)

- Ser proficiente em linguagens de programação, para manipular e analisar grandes conjuntos de dados;
- Identificar e analisar problemas, desenvolver alternativas e implementar soluções eficazes durante a execução de um projeto.



Recursos didáticos

- Acesso ao laboratório de informática e/ou internet.



Duração da aula

50 minutos.

Operação com *strings* – revisão

As *strings* suportam várias operações e métodos que permitem manipulá-las de diversas maneiras.

Nos próximos slides, vamos explorar algumas operações comuns que podem ser realizadas em *strings*:

1. Concatenação;
2. Leitura do comprimento da *string*;
3. Busca de *substrings*;
4. Fatiamento (*slicing*);
5. Transformação de maiúsculas/minúsculas;
6. Substituição de caracteres.

Operação com *strings*

1. Concatenação

Você pode juntar duas ou mais *strings* usando o operador de concatenação (+) ou simplesmente escrevendo-as uma após a outra.

```
1 str1 = "Olá, "  
2 str2 = "Mundo!"  
3 concatenated_string = str1 + str2  
4 print(concatenated_string)
```

Olá, Mundo!

```
1 print("Essa é a aula "+ 2 + " do tópico string")
```

```
-----  
TypeError                                Traceback (most recent call last)  
Cell In[44], line 1  
----> 1 print("Essa é a aula "+ 2 + " do tópico string")  
  
TypeError: can only concatenate str (not "int") to str
```

```
1 print("Essa é a aula "+ str(2) + " do tópico string")
```

Essa é a aula 2 do tópico string



Atenção!

Olha o que acontece se usarmos o operador + misturando *string* e inteiro.

Reprodução - Jupyter Notebook

Operação com *strings*

2. Leitura do comprimento da *string*

Para determinar o número de caracteres em uma *string*, você pode usar a função `len()`.

```
1 texto = "Olá, Mundo!"  
2 comprimento = len(texto)  
3 print(comprimento)
```

11

```
1 texto = ""  
2 comprimento = len(texto)  
3 print(comprimento)
```

0

Reprodução - Jupyter Notebook

Operação com *strings*

3. Busca de *substrings*

Você pode verificar se uma *substring* está presente em uma *string*, usando o operador `in`.

```
1 texto = "Olá, Mundo!"  
2 contem_ola = "Olá" in texto # Resultado: True  
3  
4 print(contem_ola)
```

True

```
1 texto = "Olá, Mundo!"  
2 contem_ola = "Ola" in texto # Resultado: True  
3  
4 print(contem_ola)
```

False

Reprodução – Jupyter Notebook

Operação com *strings*

4. Fatiamento (*slicing*)

Como mencionado anteriormente, você pode extrair partes de uma *string*, usando o *slicing*.

```
1 texto = "Olá, Mundo!"  
2 substring = texto[1:6] # Resultado: "Lá, M"  
3  
4 print(substring)
```

lá, M

Reprodução - Jupyter Notebook

Operação com *strings*

5. Transformação de maiúsculas/minúsculas

As *strings* geralmente têm métodos para transformar todos os caracteres em maiúsculas ou minúsculas.

```
1 texto = "Olá, Mundo!"
2 texto_maiusculo = texto.upper() # Resultado: "OLÁ, MUNDO!"
3 texto_minusculo = texto.lower() # Resultado: "olá, mundo!"
4
5 print(texto_maiusculo)
6 print(texto_minusculo)
```

```
OLÁ, MUNDO!
olá, mundo!
```

Reprodução – Jupyter Notebook



Reflita

Será que existe alguma função que deixe apenas a primeira letra maiúscula? E será que existe outra função que deixe a primeira letra de cada palavra maiúscula?

Operação com *strings*

6. Substituição de caracteres

Você pode substituir partes da *string* por outros caracteres, usando o método `replace()`.

```
1 texto = "Olá, Mundo!"  
2 texto_substituido = texto.replace("Olá", "Oi") # Resultado: "Oi, Mundo!"  
3 print(texto_substituido)
```

Oi, Mundo!

Reprodução - Jupyter Notebook

Strings

Essas são apenas algumas das operações que você pode realizar em *strings*. As *strings* são versáteis e há muitos outros métodos disponíveis para manipulá-las, tais como:

- dividir uma *string* em uma lista de *substrings*;
- remover espaços em branco;
- formatar *strings*, entre outros.



Saiba mais

A documentação do Python pode fornecer uma lista completa das operações e dos métodos disponíveis para manipulação de *strings*. PYTHON. *Métodos de string*. Disponível em: <https://docs.python.org/pt-br/3/library/stdtypes.html#string-methods>. Acesso em: 09 fev. 2024.



Vamos
fazer uma
atividade

A resolução dos problemas deve ser entregue pelo Ambiente Virtual de Aprendizagem (AVA).



20 minutos



Em grupo

Exercício: Manipulação de texto

Você está construindo um sistema de gerenciamento de contatos. Crie um programa que realize as seguintes tarefas:

- Peça ao usuário para digitar seu nome completo.
- Concatene "Olá," com o nome fornecido e exiba o resultado.
- Conte quantos caracteres existem no nome completo digitado e exiba o resultado.
- Peça ao usuário para digitar um sobrenome para procurar na *string* do nome completo.
- Verifique se o sobrenome fornecido está presente no nome completo e exiba uma mensagem apropriada.
- Exiba o nome completo em letras maiúsculas.
- Substitua todas as ocorrências da vogal "a" na *string* do nome completo pelo caractere '4' e exiba o resultado.



O que nós
aprendemos
hoje?

© Getty Images

Hoje desenvolvemos:

- 1** Conhecimento sobre a estrutura de dados *string*.
- 2** Conhecimento sobre alguns métodos de *string*.
- 3** Aplicação de métodos de *string*.



Saiba mais

ALURA. *Python para Data Science: primeiros passos. O que são listas?* Disponível em: <https://cursos.alura.com.br/course/python-data-science-primeiros-passos/task/123721>. Acesso em: 09 fev. 2024.

ALURA. *Python para Data Science: primeiros passos. Para saber mais: string – uma sequência de caracteres.* Disponível em: <https://cursos.alura.com.br/course/python-data-science-primeiros-passos/task/123723>. Acesso em: 09 fev. 2024.

PYTHON. *Métodos de string.* Disponível em: <https://docs.python.org/pt-br/3/library/stdtypes.html#string-methods>. Acesso em: 09 fev. 2024.

Referências da aula

MENEZES, N. N. C. *Introdução à programação com Python: algoritmos e lógica de programação para iniciantes*. São Paulo: Novatec, 2019.

Identidade visual: Imagens © Getty Images.

**Educação
Profissional
Paulista**

Técnico em
**Ciência de
Dados**